

SENTINEL - MONITOR DE AUTOMAÇÕES

Documentação Técnica, Arquitetura e Manual Operacional - v1.5

1. Visão Geral

O **Sentinel** é um sistema centralizador, leve e autônomo, desenvolvido em Python para monitorar a saúde de tarefas agendadas, rotinas batch e robôs de automação corporativos.

"Um script que não roda não consegue enviar uma mensagem avisando que não rodou."

O sistema elimina pontos cegos operacionais atuando em duas frentes complementares:

- **Notificação Ativa de Erro (Push):** Intercepta falhas em tempo de execução via decorador e envia alerta com duração e *traceback* completo da falha.
- **Dead Man's Switch (Monitor de Ausência):** Vigia contínuo em segundo plano que aguarda o sinal de vida do script. Se o script não executar dentro do horário previsto mais a tolerância, o Sentinel soa o alarme no Telegram.

2. O Conceito de Heartbeat e a Falha Silenciosa

Heartbeat (*batimento cardíaco*) é o padrão de engenharia usado para verificar se um sistema está vivo e ativo. A melhor analogia é o **monitor cardíaco de um hospital**:

- Enquanto o paciente está vivo, o monitor emite batimentos regulares: *bip... bip... bip...*
- O médico não precisa perguntar a cada segundo se o paciente vive: o batimento periódico atesta a saúde.
- Se o coração parar (silêncio), o monitor detecta a ausência e dispara a sirene de emergência: **PIIIIIIIII!**

Por que arquivos de log falham? Em falhas silenciosas (computador desligado por engano, agendador de tarefas travado, queda de energia ou erro de memória), **o script sequer chega a iniciar**. Ele não gera log nem envia e-mails. Com o Heartbeat do Sentinel, **a ausência de sinal é uma má notícia**.

3. Arquitetura e Stack de Tecnologias

Componente	Tecnologia	Responsabilidade
Backend API	FastAPI (Porta 8050)	Recebe requisições de start, success e fail em alta velocidade.
Banco de Dados	SQLite (sentinel.db)	Persistência em arquivo único com histórico e janelas esperadas.
Vigia em Background	APScheduler	Inspeciona o banco a cada 15s verificando automações ausentes.
Disparador	Telegram Bot API / Console	Entrega mensagens formatadas em HTML com blocos de código.
Dashboard Web	Tailwind CSS + Jinja2	Interface web moderna com métricas e visualizador de erros.
Clientes SDK	Python e PHP nativos	Decorador Python e wrapper PHP com identificação automática da linguagem.
MCP App	Node.js + MCP Apps	Ferramentas somente leitura e painel visual para ChatGPT, Codex e hosts MCP compatíveis.

4. Como Funciona a Função Decoradora (@monitor_job)

Um **decorador** funciona como uma "**capa protetora**" ou "**vigia**" colocado em volta da função sem alterar a regra de negócio interna. Ele adota a arquitetura clássica de 3 camadas:

1. `monitor_job(...)`: Recebe parâmetros de configuração (`job_id`, `name`, `server_url`).
2. `decorator(func)`: Recebe a função original que foi decorada (ex: `def main`).
3. `wrapper(*args, **kwargs)`: O 'dublê' que executa no lugar da sua função dentro do contexto `SentinelMonitor`.

Ciclo de Vida: No `__enter__`, avisa o Sentinel que iniciou (Executando). Se der sucesso, envia duração (OK). Se der erro, no `__exit__` extrai o *traceback* completo, dispara o alerta no Telegram e **relança a exceção original** (raise e) sem mascarar falhas.

```
from monitor import monitor_job

@monitor_job(job_id="meu_job", name="Nome da Automacao")
def main():
    try:
        processar_dados()
    except Exception as e:
        logger.error("Erro no processamento", exc_info=e)
        raise e # <- OBRIGATORIO: relancar para o Sentinel alertar!
```

4.1 Integração PHP e Identificação de Linguagem

O cliente `sentinel_monitor.php` envolve uma função PHP e registra início, sucesso, falha, duração e stack trace. Use `sentinel_fail($mensagem)` no lugar de `die($mensagem)` para preservar o detalhe da falha.

```
require_once __DIR__ . '/sentinel_monitor.php';

sentinel_monitor(
    'monitoramento_nf_email',
    'Monitoramento de notas fiscais',
    'executarMonitoramento'
);
```

O cliente Python envia `language: python` e o cliente PHP envia `language: php`. O dashboard apresenta o ícone do Python ou o elefante do PHP ao lado do job. Jobs antigos recebem a linguagem na próxima execução com o cliente atualizado.

Em scripts divididos em vários ciclos HTTP, use `return` depois de emitir o redirecionamento, em vez de `exit`. Relance erros capturados nos lotes e use `sentinel_fail(...)` se a fila não avançar ou atingir o limite de ciclos.

No retorno bancário Grafeno, o cliente pode ser importado da pasta anterior com `__DIR__ . '/../sentinel_monitor.php'`. Falhas de API, banco, download, log e cópia são monitoradas; paginação e troca de empresa terminam com `return`. Nunca inclua o header `Authorization` nem o token da API nos alertas.

5. Inicialização do Serviço e Telegram

- **Iniciar Serviço:** Duplo clique no atalho `run.bat` ou comando `python -m uvicorn app.main:app --host 127.0.0.1 --port 8050 --reload`.
- **Painel Web:** <http://127.0.0.1:8050> | **Documentação Swagger:** <http://127.0.0.1:8050/docs>
- **Telegram:** Crie o bot no **@BotFather** (/newbot), capture seu Chat ID com **@userinfobot** e configure no arquivo `.env`.

5.1 MCP App para ChatGPT e Codex

O diretório `mcp-app/` fornece um servidor MCP somente leitura. Ele consulta a API REST do Sentinel e expõe as ferramentas `sentinel_list_jobs`, `sentinel_get_executions` e `sentinel_render_status`. A última ferramenta entrega um painel visual compatível com o padrão MCP Apps; nenhuma delas cadastra, edita, remove ou envia pings.

- **Inicialização:** Execute `npm.cmd install` e `npm.cmd start` dentro de `mcp-app`, ou use `run_mcp.bat`.
- **Endpoint:** <http://127.0.0.1:8787/mcp>. Ajuste `SENTINEL_API_URL`, `MCP_HOST` e `MCP_PORT` no `.env`.
- **Segurança:** Para acesso remoto, publique por HTTPS e adicione autenticação antes de expor dados operacionais.

6. Automações Ativas Monitoradas

Job ID	Nome	Caminho do Script
Crypt_Lamina	Criptografia PDF Lâmina	C:\Automações\crypt_lamina\main.py
Slipagem_NetFactor	Slipagem NetFactor	C:\Automações\slipagem\main.py